

CONTENIDO

NIVEL FUNDAMENTAL	3
Módulo 01. Entorno y herramientas.....	3
Contenido	3
Laboratorio	3
Módulo 02. Fundamentos del Lenguaje.....	6
Contenido	6
Laboratorio	6
Resultados del Laboratorio	8
Módulo 03. Funciones y Programación “Pythonic”	9
Contenido	9
Laboratorio	9
Resultados del Laboratorio	11
Módulo 04. Objetos y Modelos de Datos	13
Contenido	13
Laboratorio	13
Resultados del Laboratorio	16
Módulo 05. Tipado Estático y Calidad	18
Contenido	18
Laboratorio	18
Resultados del Laboratorio	20
Módulo 06. Librería Estándar y E/S.....	21
Contenido	21
Laboratorio	21
Resultados del Laboratorio	23
Módulo 07. HTTP y consumo de APIs (Smocker)	24
Contenido	24
Laboratorio	24
Resultados del Laboratorio	26
NIVEL INTERMEDIO	27
Módulo 01. Pruebas y TDD	27
Contenido	27

Laboratorio	27
Resultados del Laboratorio	29
Módulo 02. Acceso a Datos y ORM.....	30
Contenido	30
Laboratorio	30
Resultados del Laboratorio	32
Módulo 03. APIs Web con FastAPI	33
Contenido	33
Laboratorio	33
Resultados del Laboratorio	35

NIVEL FUNDAMENTAL

MÓDULO 01. ENTORNO Y HERRAMIENTAS

CONTENIDO

Introducción a Python 3.12 y los ambientes virtuales para las dependencias de un proyecto, así como la herramienta Poetry para crear proyectos y gestionar ambientes virtuales. Abarca la estructura básica de un proyecto Python, guías de diseño (PEP 8 y PEP 20), y herramientas de calidad y automatización (black, isort, ruff y pre-commit).

LABORATORIO

- Debido a políticas de seguridad del equipo corporativo, no me fue posible utilizar la dependencia isort, únicamente logré su instalación. Por tanto, en este laboratorio no fue probada.
- El aprendizaje de Python supuso un reto al principio, dado que introduce diferentes comandos, herramientas y dependencias, así como un flujo de trabajo diferente al visto en Java, como la intervención continua del uso de Git Bash (en mi caso).
- Ya que este módulo representa el inicio del curso, tardé definiendo la estructura de los proyectos para tener un formato de uso práctico que mantener.

Preparación del entorno:

1. Se instaló Python 3.12.
2. Se instaló Poetry desde Bash.
3. Se agregó Poetry al PATH de usuario (variables de entorno).
4. Se configuró Poetry para crear ambientes dentro del proyecto con *virtualenvs.in-project = true*.
5. Se creó la carpeta padre **curso-python**.
6. Se creó la carpeta del 'módulo m01-entorno-herramientas'.

Creación del proyecto:

7. Se inicializó el proyecto con *poetry init*.
8. Se creó **pyproject.toml**.
9. Se configuró *package-mode = false* dentro del archivo anterior para evitar que Poetry intentara instalar el módulo como paquete.
10. Se creó el ambiente virtual mediante *poetry install*.
11. Se verificó el entorno con *poetry env info*.
12. Se comprobó Python mediante *poetry run python --version*.

Instalación de herramientas:

13. Se instalaron Black, isort, Ruff y pre-commit con *poetry add --group dev*
14. Se verificó la instalación individual de Black, Ruff y pre-commit.

15. Se confirmó la instalación de isort mediante poetry show isort.
16. Se documentó que isort no pudo ejecutarse por una política corporativa de Control de aplicaciones.

Estructura del proyecto:

17. Se crearon las carpetas src/ y tests/.
18. Se creó **README.md**.
19. Se creó **.gitignore**.
20. Se excluyeron .env/, __pycache__/, archivos compilados y caches locales.

Configuración de herramientas:

21. Se configuró Black en **pyproject.toml**.
22. Se definió la longitud de línea y la versión objetivo py312.
23. Se configuró isort con el perfil compatible con Black.
24. Se configuró Ruff para revisar estilo, errores e imports.
25. Se definió Ruff con **target-version = "py312"**.

Prueba de formato y análisis:

26. Se creó **src/lejemplo.py**.
27. Se agregaron infracciones intencionales de estilo e imports para el uso de las dependencias.
28. Se ejecutó Black mediante **poetry run black src**.
29. Black corrigió el formato del archivo.
30. Se ejecutó Ruff mediante **poetry run ruff check src**.
31. Se ejecutó Ruff con **--fix** para aplicar correcciones automáticas.
32. isort no se ejecutó debido al **bloqueo** del equipo.

Configuración de Git:

33. Se inicializó un repositorio local con **git init**.
34. Se configuraron el nombre y correo de Git.
35. Se creó **.pre-commit-config.yaml**.
36. Se agregaron hooks para Black, Ruff y Ruff Format.
37. isort se excluyó temporalmente de pre-commit.
38. Se instaló el hook local con **poetry run pre-commit install**.

Ejecución de pre-commit:

39. Se ejecutaron los **hooks** sobre todos los archivos *con poetry run pre-commit run --all-files*.
40. Se identificó el aviso de archivos sin preparar.
41. Se ejecutó Black manualmente para evitar conflictos con cambios no registrados.
42. Se actualizaron los archivos mediante *git add ..*
43. Se repitió la validación hasta que los **hooks** pasaron sin modificar archivos.
44. Se creó el commit local del módulo.

Documentación final:

45. Se actualizó **README.md**.
46. Se documentó el objetivo del módulo y las herramientas utilizadas.
47. Se registró la excepción local de isort.
48. Se aclaró que isort permanece instalado y configurado, pero pendiente de autorización.
49. Se documentó el uso provisional de Ruff para revisar imports.
50. Se registró la estructura final del proyecto.
51. Se verificó que **.venv/** no se versionara.
52. Se confirmó que los cambios quedaron guardados en Git.

Resultados:

53. Se creó un proyecto Python administrado con Poetry.
54. Se configuró un ambiente virtual aislado.
55. Se configuraron herramientas de formato, análisis y automatización.
56. Se aplicaron PEP 8 y criterios de legibilidad.
57. Se automatizaron validaciones mediante pre-commit.
58. Se documentó una excepción local causada por las políticas de seguridad corporativas.

MÓDULO 02. FUNDAMENTOS DEL LENGUAJE

CONTENIDO

Abarca conceptos fundamentales del lenguaje Python como sintaxis, indentación y variables. Así mismo contiene tipos básicos (variables) y colecciones de Python (listas, tuplas, diccionarios y conjuntos), controles de flujo (ciclos, match-case), errores y excepciones (try-except), expresiones regulares y módulos estándar (re, json), y lectura de archivos JSON.

LABORATORIO

- Durante su elaboración, presenté dificultades para comprender y aplicar los conceptos: colecciones, manejo de errores y excepciones, módulo estándar “re”, y el manejo de archivos JSON.
- Quise implementar las herramientas del módulo anterior (black, ruff y pre-commit), por consideración propia para seguir practicando, lo cual aplazó el tiempo dedicado para el módulo.

Preparación del proyecto:

1. Se creó la carpeta ‘m02-fundamentos-del-lenguaje’ dentro de curso-python.
2. Se inicializó el proyecto con Poetry.
3. Se configuró `package-mode = false` en **pyproject.toml**.
4. Se creó el ambiente virtual con *poetry install*.
5. Se instalaron Black, isort, Ruff y pre-commit como dependencias de desarrollo.
6. Se crearon las carpetas src/ y datos/.

Archivo de entrada:

7. Se creó *datos/usuarios.json*.
8. Se agregaron usuarios con nombre, correo, rol y estado activo.
9. Se verificó el contenido del archivo desde VS Code y Git Bash.

Script principal:

10. Se creó **src/main.py**.
11. Se importaron json, re y Path.
12. Se definió `RUTA_DATOS = Path(__file__).parent.parent / ...` para localizar el JSON.
13. Se definió `PATRON_CORREO = ...` para validar correos.

Lectura y validación del JSON:

14. Se creó la ruta `cargar_usuarios`.
15. Se abrió el archivo mediante `with ruta.open(...)`.

16. Se convirtió el contenido mediante `json.load(archivo)`.
17. Se manejó excepción `FileNotFoundError` para archivos inexistentes.
18. Se manejó excepción `json.JSONDecodeError` para JSON inválido.
19. Se utilizó `raise ... from error` para conservar la causa original.
20. Se validó que el JSON contuviera una lista.
21. Se utilizó `TypeError` cuando la estructura recibida no tenía el tipo esperado.

Validación y filtrado:

22. Se creó `correo_es_valido(correo)`.
23. Se validó el correo mediante `re.fullmatch(...)`.
24. Se creó `filtrar_usuarios(usuarios)`.
25. Se descartaron elementos que no fueran diccionarios.
26. Se filtraron únicamente usuarios con "activo": true.
27. Se descartaron correos ausentes, no textuales o inválidos.
28. Se conservaron los usuarios válidos en una lista.

Agregación y salida:

29. Se creó `contar_por_rol(usuarios)`.
30. Se contó cada rol mediante un diccionario y `.get(...)`.
31. Se actualizó `main()` para coordinar la carga, el filtrado y la agregación.
32. Se mostró la lista de usuarios válidos.
33. Se mostró el resumen de usuarios por rol.
34. Se protegió la ejecución con `if __name__ == "__main__": ....`

Pruebas funcionales:

35. Se probó el caso normal con un JSON válido.
36. Se probó un archivo inexistente.
37. Se probó un JSON con formato inválido.
38. Se probó un JSON válido con una estructura incorrecta.
39. Se restauró el archivo JSON original después de las pruebas.

Calidad y automatización:

40. Se ejecutó Black para formatear el código.
41. Se ajustó `target-version` a py312 en `pyproject.toml`.
42. Se ejecutó Ruff para analizar el código.

43. Se utilizó Ruff con `--fix` para correcciones automáticas compatibles.
44. Se corrigió manualmente el uso de `ValueError` por `TypeError` en la validación de estructura, el cuál se detectó con Ruff.
45. Se creó `.pre-commit-config.yaml`.
46. Se configuraron `hooks` para Black, Ruff y Ruff Format.
47. Se inicializó un repositorio Git local.
48. Se instaló el hook mediante `poetry run pre-commit install`.
49. Se ejecutaron las validaciones con `poetry run pre-commit run --all-files`.
50. Se creó el commit final del laboratorio.

Documentación y limpieza:

51. Se creó o actualizó `README.md` con el objetivo, flujo, pruebas y excepciones del laboratorio.
52. Se agregó `.ruff_cache/` al `.gitignore`.
53. Se documentó que `isort` está instalado, pero bloqueado por una política corporativa.
54. Se verificó que `.venv/`, `.ruff_cache/` y `__pycache__/` no se versionen.
55. Se confirmó que el repositorio quedó con el trabajo registrado.

RESULTADOS DEL LABORATORIO

El script `'src/main.py'` realiza las siguientes operaciones:

1. Construye la ruta del archivo JSON mediante `pathlib.Path`.
2. Abre el archivo usando un bloque `with`.
3. Convierte el contenido JSON a estructuras Python mediante `json.load`.
4. Maneja `FileNotFoundError` cuando el archivo no existe.
5. Maneja `json.JSONDecodeError` cuando el contenido no es un JSON válido.
6. Valida que el JSON contenga una lista.
7. Filtra usuarios activos.
8. Valida el formato de los correos con `re.fullmatch`.
9. Descarta registros incompletos o con correo inválido.
10. Agrupa los usuarios válidos por rol.
11. Muestra los usuarios filtrados y un resumen por rol.

MÓDULO 03. FUNCIONES Y PROGRAMACIÓN “PYTHONIC”**CONTENIDO**

Cubre las funciones y diseños de APIs, donde define las funciones, parámetros y argumentos (posicionales, nombrados y variables). La definición y uso de lambdas, closures y decoradores, así como su sintaxis (@) y funciones específicas. Qué son los iteradores, generadores y comprensiones. Qué son los context managers y cómo utilizarlos.

LABORATORIO

- Tuve dificultades para comprender el uso de nuevas funciones destinadas a este laboratorio, como el uso de funciones para conservar metadatos o las capturas de errores, dado los diferentes tipos que hay.
- Nuevamente implementé herramientas de módulos anteriores, lo que me permitió mejorar con la práctica y reforzar conocimientos.

Preparación del proyecto:

1. Se creó la carpeta 'm03-pythonic' dentro de curso-python.
2. Se abrió el proyecto en VS Code.
3. Se seleccionó el intérprete **.venv/Scripts/python.exe**.
4. Se inicializó el proyecto con Poetry.
5. Se configuró `package-mode = false` en **pyproject.toml**.
6. Se configuraron Black, isort y Ruff para Python 3.12.
7. Se creó el ambiente virtual con **poetry install**.
8. Se crearon src/, README.md y .gitignore.
9. Se agregaron al .gitignore .venv/, __pycache__/, archivos compilados y cachés locales.
10. Se instalaron Black, isort, Ruff y pre-commit como dependencias de desarrollo.
11. Se verificaron las herramientas instaladas.
12. Se documentó que isort permanece bloqueado por la política corporativa.

Generador por lotes:

13. Se creó **src/loter.py**.
14. Se implementó **generar_lotes(datos, tamaño)**.
15. Se validó que tamaño sea un entero.
16. Se rechazó bool como tamaño válido.
17. Se validó que tamaño sea mayor que cero.
18. Se usó **yield** para producir lotes progresivamente.
19. Se probaron datos con tamaño exacto.

20. Se probaron datos con un último lote incompleto.

21. Se probó una colección vacía.

22. Se probaron tamaños inválidos.

Context manager de temporización:

23. Se creó *src/temporizador.py*.

24. Se utilizó `@contextmanager`.

25. Se midió el tiempo con `perf_counter`.

26. Se empleó `try-finally`.

27. Se verificó que el tiempo se muestre al terminar el bloque.

28. Se comprobó que `finally` se ejecute también ante una *excepción*.

Decorador de reintentos:

29. Se creó *src/reintentos.py*.

30. Se implementó un decorador configurable.

31. Se usaron `*args` y `**kwargs`.

32. Se utilizaron `functools.wraps`.

33. Se definieron los intentos máximos.

34. Se definieron el tiempo inicial de espera.

35. Se implementaron ***backoff*** progresivo.

36. Se limitaron las excepciones capturadas.

37. Se relanzaron el último error si se agotan los intentos.

38. Se probaron una función que falla y después funciona.

39. Se probaron una función que siempre falla.

Integración:

40. Se creó *src/main.py*.

41. Se generaron datos de ejemplo.

42. Se procesaron mediante el generador por lotes.

43. Se midió el procesamiento con el context manager.

44. Se envolvieron una operación inestable con el decorador de reintentos.

45. Se ejecutaron el flujo completo.

46. Se comprobaron los resultados y mensajes.

Calidad del código:

47. Se ejecutaron las dependencias Black y Ruff.
48. Se corrigieron manualmente los problemas que Ruff no pueda resolver.
49. isort permaneció instalado y configurado, pero sin ejecución directa.
50. Se creó *pre-commit-config.yaml*.
51. Se configuraron hooks para Black, Ruff y Ruff Format.
52. Se inicializó Git local.
53. Se instaló el hook con *poetry run pre-commit install*.
54. Se ejecutaron los *hooks* sobre todos los archivos.
55. Se corrigieron cambios automáticos antes de preparar el commit.

Documentación y cierre:

56. Se actualizó *README.md*.
57. Se documentaron el generador, el decorador y el context manager.
58. Se incluyeron los comandos de ejecución y validación.
59. Se documentaron la excepción de isort.
60. Se agregó el *.gitignore*.
61. Se ejecutó el programa final.
62. Se comprobó con git status.
63. Se creó el commit final del laboratorio.
64. Se verificó que el repositorio quede limpio.

RESULTADOS DEL LABORATORIO

Archivos principales:

- *src/lotes.py*: generador que divide una secuencia en lotes.
- *src/temporizador.py*: context manager para medir la duración de un bloque.
- *src/reintentos.py*: decorador configurable para reintentar operaciones.
- *src/main.py*: integración de los componentes.
- *pyproject.toml*: configuración del proyecto y sus herramientas.
- *.pre-commit-config.yaml*: hooks automáticos de calidad.
- *.gitignore*: archivos y carpetas excluidos de Git.

El flujo principal:

1. Crea una colección de datos de ejemplo.
2. Genera lotes de tamaño definido.
3. Procesa cada lote.
4. Simula un fallo temporal.
5. Reintenta automáticamente la operación.
6. Mide el tiempo total de procesamiento.

MÓDULO 04. OBJETOS Y MODELOS DE DATOS

CONTENIDO

Definición de clases, objetos, métodos y atributos. Conceptos herencia, sobreescritura, composición, y métodos especiales 'dunder methods'. Uso de los `dataclass` y `attrs` para generación de código automático para representar datos, así como sus configuraciones. Cómo funciona la biblioteca Pydantic.

LABORATORIO

- El uso de los diferentes dunder methods me generó confusión en un principio dado que no diferenciaba qué hacía cada método o cómo debían ser aplicados.
- La biblioteca Pydantic cuenta con varias funcionalidades por encima de las `dataclass`, pero comprender su estructura para usarlas se me dificultó.
- Comprendí fácilmente el flujo de la ejecución del proyecto, pero el uso del contenido de este módulo para cada código fue complicada dadas las diferentes validaciones, métodos, herencias, composiciones y exclusiones (de atributos).

Preparación del proyecto:

1. Se creó la carpeta `m04-modelos-datos` dentro de `curso-python`.
2. Se abrió el proyecto en VS Code y se inicializó el proyecto con Poetry.
3. Se configuró `package-mode = false` en `pyproject.toml`.
4. Se configuraron Black, isort y Ruff para Python 3.12.
5. Se ejecutó `poetry install`.
6. Se seleccionó el intérprete `.venv/Scripts/python.exe`.
7. Se crearon `src/`, `README.md` y `.gitignore`.
8. Se excluyeron `.venv/`, `__pycache__`, archivos compilados y cachés.

Se instaló **Pydantic**.

9. Se instalaron Black, isort, Ruff y pre-commit.
10. Se verificaron las versiones y dependencias.

Entidad **Order**:

11. Se creó `src/entidades.py`.
12. Se definió `Order` con `@dataclass`.
13. Se agregaron los atributos principales: `order_id`, `producto`, `unit_price`, `quantity` y `tax_rate`.
14. Se definieron los **campos calculados** con `field(init=False)`.
15. Se implementó `__post_init__`.

16. Se calcularon `subtotal`, `tax` y `total`.
17. Se implementó `__str__`.
18. Se implementó `__eq__` comparando `order_id`.
19. Se implementó `__lt__` comparando `total`.
20. Se probaron los cálculos y las comparaciones.

Modelos Pydantic:

21. Se creó **`src/modelos.py`**.
22. Se definió `OrderIn`.
23. Se definieron restricciones con `Field`.
24. Se validaron `order_id` y `product`.
25. Se validaron `unit_price`, `quantity` y `tax_rate`.
26. Se implementó un validador para textos vacíos.
27. Se definió `OrderOut`.
28. Se incluyeron los campos calculados en `OrderOut`.
29. Se probó la creación de modelos válidos.
30. Se probaron entradas inválidas.
31. Se comprobó la conversión de tipos de *Pydantic*.
32. Se probó `model_dump()`.
33. Se probó `model_dump_json()`.

Conversión e integración:

34. Se creó **`src/main.py`**.
35. Se implementó `order_in_to_entity(order_in)`.
36. Se implementó `order_to_order_out(order)`.
37. Se conectó el flujo: `OrderIn` → `Order` → `OrderOut`
38. Se validaron los datos antes de crear la entidad.
39. Se calcularon los valores derivados en `Order`.
40. Se serializó la salida como diccionario y JSON.
41. Se protegió la ejecución con `if __name__ == "__main__": ....`

Pruebas:

42. Se capturó `ValidationError`.
43. Se probaron datos inválidos:

- Identificador vacío.
- Producto vacío.
- Precio negativo.
- Cantidad igual a cero.
- Impuesto fuera del rango.

44. Se probaron comparaciones entre órdenes.

45. Se probó igualdad por `order_id`.

46. Se probó comparación por total.

47. Se verificaron las salidas de `model_dump()` y `model_dump_json()`.

Calidad y Git (módulo 01):

48. Se ejecutaron Black y Ruff sobre `src/`.

49. Se aplicaron correcciones automáticas compatibles.

50. Se creó **`.pre-commit-config.yaml`**.

51. Se configuraron hooks para Black, Ruff y Ruff Format.

52. `isort` permaneció instalado y configurado, pero sin ejecución por el bloqueo corporativo.

53. Se inicializó Git local.

54. Se instaló el hook con `poetry run pre-commit install`.

55. Se ejecutaron los hooks sobre todos los archivos.

56. Se corrigieron cambios automáticos antes del commit.

57. Se creó el commit del laboratorio.

Documentación y cierre:

58. Se actualizó **`README.md`**.

59. Se documentaron la entidad, los modelos y las conversiones.

60. Se documentaron las validaciones y serializaciones.

61. Se documentó la excepción local de `isort`.

62. Se verificó la estructura del proyecto.

63. Se confirmó que las carpetas locales no se versionaran.

64. Se comprobó el estado final del repositorio.

65. Se confirmó que el laboratorio quedó completo.

RESULTADOS DEL LABORATORIO

Archivos principales

- **src/entidades.py**: contiene la entidad **Order**.
- **src/modelos.py**: contiene los modelos **Pydantic** **OrderIn** y **OrderOut**.
- **src/main.py**: coordina la validación, conversión, comparación y serialización.
- **pyproject.toml**: contiene la configuración del proyecto y sus herramientas.
- **.pre-commit-config.yaml**: define los hooks automáticos.
- **.gitignore**: excluye archivos y carpetas locales.

Métodos especiales que implementa la entidad **Order**:

- **__str__** Proporciona una representación legible.

```
Order ORD-001: Teclado - Total: 116.00
```
- **__eq__** Define la igualdad entre órdenes utilizando **order_id**. Dos órdenes con el mismo identificador se consideran iguales, aunque tengan otros datos diferentes.
- **__lt__** Permite utilizar el operador `<` para comparar órdenes según su 'total'.

Ejemplo:

```
order_a < order_b
```

El flujo principal del laboratorio es:

```
Datos externos
  ↓
OrderIn
  ↓ validación
Order
  ↓ cálculos derivados
OrderOut
  ↓ serialización
Diccionario o JSON
```

Se implementaron:

- ✓ La entidad **Order** como **dataclass**.
- ✓ Cálculos derivados de subtotal, impuesto y total.
- ✓ Comparación de órdenes por identificador y total.
- ✓ El modelo de entrada **OrderIn**.
- ✓ El modelo de salida **OrderOut**.

- ✓ Validación de datos con **Pydantic**.
- ✓ Conversión de modelos **Pydantic a entidad**.
- ✓ Conversión de entidad a modelo de salida.
- ✓ Serialización a diccionario y JSON.
- ✓ Validaciones de calidad con Black, Ruff y pre-commit.
- ✓ Documentación del proyecto y de la excepción de isort.

MÓDULO 05. TIPADO ESTÁTICO Y CALIDAD

CONTENIDO

Type hints básicos: anotaciones de variables, parámetros, colecciones tipadas, validaciones. Tipado avanzado: **Union** para tipos de valor, **Literal** para opciones concretas, **TypedDict** para dar estructura a diccionarios, y **Protocol** para definir comportamientos. Verificación estática con *mypy* y *pyright*. Guías de calidad y estilo con PEP 8, black, ruff, isort y PEP 20. Automatización con pre-commit para uso local y CI en repositorios remotos.

LABORATORIO

- Los comandos para el tipado avanzado resultaron un poco confusos al no comprender en un principio su uso específico, y después por cómo debían usarse (respecto a su estructura).
- Las guías de calidad y estilo, así como las herramientas automatización fueron sencillas de aprender, dado que es lo que he visto desde el primer módulo. Únicamente se sumo CI para comprenderlo a más detalle, pero ver antes pre-commit lo facilitó.

Preparación del proyecto:

1. Se creó la carpeta m05-tipado-calidad dentro de curso-python.
2. Se abrió el proyecto en VS Code e inicializó el proyecto con Poetry.
3. Se configuró `package-mode = false` en *pyproject.toml*.
4. Se configuraron Black, isort y Ruff para Python 3.12.
5. Se ejecutó *poetry install*.
6. Se seleccionó el intérprete `.venv/Scripts/python.exe`.
7. Se crearon `src/`, `README.md` y `.gitignore`.
8. Se excluyeron `.venv/`, `cache`s y archivos compilados.
9. Se instalaron *Pydantic*, Black, isort, Ruff, pre-commit y *mypy*.
10. Se verificaron las herramientas instaladas.

Incorporación y anotación del código:

11. Se reutilizó el código del Módulo 04 como base de trabajo (Ver secciones 'Entidad Order' a 'Conversión e integración' para revisar el contenido reutilizado).
12. Se corrigió la comparación poco útil entre Order y OrderOut.
13. Se agregó una comparación válida entre dos entidades Order.

Configuración y ejecución de mypy:

14. Se instaló *mypy* como *dependencia de desarrollo*.
15. Se ejecutó mypy sobre **src** (la primera ejecución terminó sin errores).

16. Se configuró mypy en **pyproject.toml**., con Python 3.12 como versión objetivo.
17. Se activó `strict = true`.
18. Se configuró `mypy_path = "src"`.
19. Se ejecutó nuevamente mypy.
20. Se confirmó la salida: `Success: no issues found in 3 source files`

Calidad de código:

21. Se ejecutó Black sobre **src**.
22. Se comprobó el formato con `black --check`.
23. Se ejecutó Ruff con las correcciones automáticas compatibles.
24. Se volvió a ejecutar Ruff para confirmar que no quedaran problemas.
25. isort permaneció instalado y configurado, pero no se ejecutó por el bloqueo corporativo.

Automatización con pre-commit:

26. Se creó **.pre-commit-config.yaml**.
27. Se configuraron **hooks** para Black, Ruff y Ruff Format.
28. Se agregó **mypy** a los hooks.
29. Se añadió **Pydantic** como dependencia adicional del entorno de mypy cuando fue necesario.
30. Se inicializó el repositorio Git local.
31. Se instaló el hook mediante `poetry run pre-commit install`.
32. Se ejecutaron todos los hooks con `pre-commit run --all-files`.
33. Se corrigieron y prepararon los archivos modificados por los hooks.
34. Se repitieron las validaciones hasta que todos los hooks pasaron.

Configuración de CI:

35. Se creó **.github/workflows/quality.yml**.
36. Se configuró **GitHub Actions**.
37. Se definió Python 3.12 como versión del workflow.
38. Se configuró la instalación de **Poetry** y de las dependencias.
39. Se agregaron checks para mypy y Ruff.
40. Se agregó la comprobación de formato con Black.
41. Se aclaró que CI requiere un repositorio remoto para ejecutarse.

Documentación y cierre

42. Se actualizó **README.md**.
43. Se documentaron los *type hints* aplicados.
44. Se documentó la configuración de *mypy*.
45. Se documentaron Ruff, Black y pre-commit.
46. Se documentó el *workflow de CI*.
47. Se registró la excepción local de isort.
48. Se verificó la estructura del proyecto.
49. Se ejecutaron las validaciones finales.
50. Se prepararon los cambios con `git add ..`
51. Se creó el commit del laboratorio.
52. Se verificó el estado final del repositorio.
53. Se confirmó que el laboratorio quedó completo.

RESULTADOS DEL LABORATORIO

El workflow utiliza GitHub Actions y ejecuta:

1. Checkout del repositorio.
2. Configuración de Python 3.12.
3. Instalación de Poetry.
4. Instalación de dependencias.
5. Ejecución de *mypy*.
6. Ejecución de Ruff.
7. Comprobación de Black.

El *check* se ejecuta en eventos de *push* y *pull_request*.

En este módulo se logró:

- Anotar el código con tipos.
- Configurar mypy en *modo estricto*.
- Verificar el código estáticamente.
- Aplicar Black y Ruff.
- Automatizar validaciones con pre-commit.
- Crear un check básico de CI.
- Documentar la excepción local de isort.

MÓDULO 06. LIBRERÍA ESTÁNDAR Y E/S**CONTENIDO**

Uso de **pathlib** para el manejo de rutas y archivos. Archivos con formatos de tipo CSV, JSON y YAML; usos compatibles con Python. Uso de zonas horarias con **datetime**. Cómo funciona **logging** para el registro de eventos, configuración, niveles, handlers y diferencias con **print**. Módulo de **subprocess** para la ejecución de comandos externos desde Python.

LABORATORIO

- La estructura y uso de **pathlib** me fue complicada de entender por las diferentes acciones que incluye el paquete.
- La diferencia de uso en los formatos si bien es diferente, se me dificultó el manejo de cada uno para Python porque es diferente, siendo CSV el que sobresale.
- El uso de zonas horarias fue intuitivo, únicamente tuve confusión en los comandos para manejar los datos según sea la información deseada.

Preparación del proyecto:

1. Se creó la carpeta **m06-libreria-estandar-io** dentro de **curso-python**.
2. Se abrió el proyecto en VS Code y inicializó el proyecto con Poetry.
3. Se realizaron las configuraciones pertinentes para las dependencias, intérprete, README y **.gitignore**; como se ha visto en módulos anteriores.
4. Se crearon las carpetas **src/**, **datos/**, **salidas/** y **logs/**.
5. Se excluyeron ambientes virtuales, cachés, archivos compilados y logs generados.
6. Se instalaron las herramientas de desarrollo ya utilizadas anteriormente (configuración del Módulo 1).

Archivo de entrada:

7. Se creó **datos/ventas.csv**.
8. Se definieron las columnas **fecha**, **producto**, **cantidad** y **precio**.
9. Se agregaron cinco registros de ventas.
10. Se utilizaron fechas con **zona horaria UTC**.

Ingesta del CSV:

11. Se creó **src/ingesta.py**.
12. Se construyeron rutas con **pathlib.Path**.
13. Se leyó el CSV mediante **csv.DictReader**.
14. Se convirtieron los valores de cantidad a **int**, y precio a **float**.
15. Se convirtieron las fechas mediante **datetime.fromisoformat**.
16. Se almacenaron las filas procesadas en una lista de diccionarios.

17. Se añadieron anotaciones de tipo (Módulo 5).
18. Se verificó la lectura correcta de las cinco filas.

Logging y manejo de errores

19. Se creó un logger por módulo.
20. Se configuraron niveles `DEBUG`, `INFO`, `WARNING` y `ERROR`.
21. Se registró el inicio y el final de la lectura.
22. Se registraron detalles de filas procesadas en nivel `DEBUG`, y filas inválidas en nivel `WARNING`.
23. Se registraron errores de archivo mediante `logger.exception`.
24. Se probó el caso de archivo inexistente.
25. Se probó una cantidad no numérica.
26. Se probó una fecha inválida.
27. Se verificó que las filas inválidas se descartaran sin detener todo el procesamiento.

Cálculo de métricas:

28. Se creó **`src/metricas.py`**.
29. Se definió `Metricas` mediante `TypedDict` (Módulo 5).
30. Se implementó `calcular_metricas(ventas)`.
31. Se calculó la cantidad de ventas, unidades vendidas, ingreso de cada fila y el ingreso total.
32. Se identificó el producto con mayor ingreso.
33. Se corrigió el uso de `max` para satisfacer `mypy` (Módulo 5).

Exportación a JSON:

34. Se creó **`src/exportacion.py`**.
35. Se creó automáticamente la carpeta de salida con `Path.mkdir`.
36. Se exportaron las métricas mediante `json.dump`.
37. Se utilizó UTF-8.
38. Se aplicó formato legible con `indent=2`.
39. Se registraron errores de escritura.
40. Se generó `salidas/metricas.json`.

Integración

41. Se creó **`src/main.py`**.
42. Se configuró logging para consola y archivo.

43. Se creó **logs/ventas.log**.
44. Se integraron las funciones de ingesta, métricas y exportación.
45. Se gestionaron las rutas desde la raíz del proyecto.
46. Se ejecutó el flujo completo.
47. Se verificó la generación del JSON y del archivo de logs.
48. Se comprobó que el historial de logs se conservara entre ejecuciones.

Validación y cierre

49. Se validó el código con las herramientas configuradas anteriormente (Módulo 1 y Módulo 5), comprobando Black, Ruff, mypy y pre-commit.
50. isort permaneció instalado y configurado, pero no se ejecutó por el bloqueo corporativo ya documentado.
51. Se actualizó **README.md** con el flujo propio del Módulo 6.
52. Se documentaron las métricas, el logging, el CSV, el JSON y los errores manejados.
53. Se creó el commit final del laboratorio.
54. Se verificó el estado limpio del repositorio.

RESULTADOS DEL LABORATORIO

Se implementó un flujo completo que:

- ✓ Lee ventas desde CSV.
- ✓ Convierte los datos a tipos adecuados.
- ✓ Interpreta fechas con zona horaria.
- ✓ Calcula métricas.
- ✓ Exporta resultados a JSON.
- ✓ Registra eventos en consola y archivo.
- ✓ Maneja errores de archivo y datos inválidos.
- ✓ Utiliza rutas seguras con [pathlib](#).

MÓDULO 07. HTTP Y CONSUMO DE APIS (SMOCKER)**CONTENIDO**

Consumo de APIs HTTP con `requests`, `aiohttp` y `httpx`, incluyendo clientes síncronos y asíncronos, HTTP/2 y reutilización de conexiones. Configuración de timeouts, reintentos, backoff y manejo de errores. Streaming de respuestas mediante bloques, escritura binaria y archivos temporales. Uso de Smocker y Docker para simular APIs y probar clientes HTTP robustos.

LABORATORIO

- Tuve dificultades en la configuración de Smocker, pues inicialmente utilicé métodos que la versión de uso no permitía (PUT cambió por POST). Únicamente aceptaba GET, POST y OPTIONS.
- La estructura del archivo de mocks no coincidía con el formato esperado por Smocker, por lo que realicé los ajustes necesarios.
- Siguiendo con Smocker, al intentar realizar algunas correcciones mediante el método DELETE, no lo permitía. Por tanto, limpié mocks utilizando el endpoint /reset mediante POST.
- El uso de mypy requirió de agregar la dependencia `httpx[http2]` a su hook.

Preparación del proyecto:

1. Se creó `m07-http-apis`, se inicializó Poetry, se configuró `package-mode = false`, se añadieron las configuraciones de Black, Ruff y mypy, y se creó el ambiente virtual con Python 3.12.
2. Se seleccionó `.venv/Scripts/python.exe` en VS Code y se verificó el intérprete activo.
3. Se crearon `src/`, `descargas/`, `README.md` y `.gitignore`, excluyendo el ambiente, cachés y archivos generados.
4. Se instaló `httpx[http2]` junto con las herramientas de calidad utilizadas anteriormente (configuración de los Módulos 1 y 5).

Cliente HTTP y timeouts:

5. Se creó `src/cliente_http.py`.
6. Se configuró `httpx.Client` con HTTP/2, timeouts explícitos y encabezado User-Agent.
7. Se creó un `context manager` para abrir y cerrar correctamente el cliente.
8. Se implementó `realizar_get` para ejecutar solicitudes, validar respuestas y registrar información HTTP.
9. Se añadió el manejo de errores temporales mediante `ErrorHTTPReintenable`.
10. Se configuró `validar_respuesta` para considerar reintentables las respuestas **429** y **5xx**.

Reintentos y backoff:

11. Se implementó un **decorador de reintentos** con ParamSpec, TypeVar, *args, **kwargs y functools.wraps (conceptos del Módulo 3).
12. Se configuraron intentos máximos, espera inicial y factor de backoff.
13. Se añadieron reintentos para **TimeoutException**, **ConnectError** y errores HTTP temporales.
14. Se verificó que la excepción final se relanzara al agotarse los intentos.
15. Se corrigieron las anotaciones del decorador para que **mypy** funcionara en modo estricto (configuración del Módulo 5).

Streaming y descarga:

16. Se creó **src/descarga.py**.
17. Se implementó **archivo_temporal** mediante un **context manager** (contenido del Módulo 3).
18. Se utilizó **cliente.stream** e **iter_bytes** para procesar respuestas por bloques.
19. Se escribieron los datos en modo binario mediante archivos .part.
20. Se validó la respuesta antes de escribir el contenido.
21. Se eliminó el archivo parcial ante errores.
22. Se renombró el archivo temporal al destino final después de una descarga exitosa.
23. Se aplicaron reintentos también al flujo de streaming.

Smocker y Docker:

24. Se creó **docker-compose.yml** y se fijó **Smocker** en la versión **0.18.5**.
25. Se iniciaron y verificaron los servicios mediante **Docker Compose**.
26. Se confirmó que el **puerto 8081** correspondía a la interfaz administrativa y el 8080 al servidor simulado.
27. Se crearon y registraron **mocks** para GET /archivo.bin, GET /ejemplo.bin y GET /error.bin.
28. Se corrigió el formato de carga de mocks enviando un arreglo JSON directamente a /mocks.
29. Se identificó que el código 666 de Smocker significa que no existe un mock coincidente.
30. Se utilizaron los **endpoints de Smocker** para consultar mocks y restablecer el estado mediante POST /reset.

Integración y pruebas:

31. Se creó **src/main.py**.
32. Se integraron cliente HTTP, timeouts, reintentos, streaming y escritura segura.

33. Se probó una descarga exitosa desde Smocker.
34. Se probó un *error de conexión* usando el puerto 9999.
35. Se verificó el *backoff* de 0.5 y 1.0 segundos.
36. Se comprobó que la conexión fallida terminara relanzando `httpx.ConnectError`.
37. Se probó una *respuesta 503* para activar reintentos HTTP temporales.
38. Se restauró la descarga exitosa desde `/archivo.bin`.
39. Se verificó que no quedaran archivos `.part`.

Validación y cierre:

40. Se validó el código con las herramientas de calidad ya estudiadas (Módulos 1 y 5), comprobando Black, Ruff, mypy y pre-commit.
41. Se añadió `httpx[http2]` como dependencia adicional del *hook de mypy* para su ambiente aislado.
42. isort permaneció instalado y configurado, pero no se ejecutó por el bloqueo corporativo documentado.
43. Se actualizó README.md con el cliente HTTP, resiliencia, streaming, Smocker y Docker Compose.
44. Se creó el commit final del laboratorio.
45. Se verificó que el repositorio quedara en estado limpio.

RESULTADOS DEL LABORATORIO

Se implementó un cliente HTTP robusto capaz de:

- ✓ Consumir endpoints simulados.
- ✓ Configurar HTTP/2 y timeouts.
- ✓ Reintentar errores temporales.
- ✓ Aplicar backoff progresivo.
- ✓ Manejar errores de red y HTTP.
- ✓ Descargar respuestas grandes por streaming.
- ✓ Escribir de forma incremental y segura.
- ✓ Evitar archivos finales incompletos.

NIVEL INTERMEDIO

MÓDULO 01. PRUEBAS Y TDD

CONTENIDO

Desarrollo guiado por pruebas mediante el ciclo *Red-Green-Refactor* con pytest. Uso de fixtures, parametrización y markers para organizar suites confiables. Mocking con `unittest.mock` para aislar dependencias externas. Pruebas basadas en propiedades con *Hypothesis*, generación automática de casos y shrinking. Medición de cobertura con `pytest-cov` e integración de pruebas, calidad y umbrales en CI.

LABORATORIO

- Inicialmente, pytest no encontraba el paquete **src**, dado que no identificaba el uso del punto "." como una ruta válida. Se creó **src/__init__.py** y se configuró `pythonpath = ["."]` en **pyproject.toml**.
- Los datos de prueba se inferían como `list[dict[str, object]]`. Se anotaron explícitamente como `list[Venta]`.
- Mypy no encontraba *pytest* ni *hypothesis*. Se agregaron como dependencias adicionales del hook de mypy.

Preparación del proyecto:

1. Se creó *intermedio/m01-pruebas-tdd*, se inicializó Poetry, se configuró `package-mode = false`, se añadieron configuraciones de pytest, cobertura, Black, Ruff y mypy, y se creó el ambiente virtual con Python 3.12.
2. Se seleccionó `.venv/Scripts/python.exe` en VS Code, se crearon `src/`, `tests/`, `.github/workflows/`, `README.md` y `.gitignore`, y se instalaron pytest, *Hypothesis*, *pytest-cov*, mypy, Black, Ruff y pre-commit (configuración de calidad de módulos anteriores).
3. Se agregó **src/__init__.py** y se configuró `pythonpath = ["."]` para permitir que pytest encontrara el paquete `src`.

Desarrollo mediante TDD:

4. Se creó **tests/test_ventas.py** y se escribió la primera prueba para una lista de ventas vacía.
5. Se ejecutó pytest en estado *Red* y se corrigió el problema inicial de importación de `src`.
6. Se creó **src/ventas.py** y se implementó el código mínimo para que la primera prueba pasara.
7. Se agregó una prueba para ventas válidas y se implementó el cálculo de cantidad e ingreso total.
8. Se añadieron pruebas parametrizadas para cantidades y precios iguales o menores que cero.

9. Se implementaron validaciones que lanzan `ValueError` para ventas inválidas.

Fixtures, markers y mocking:

10. Se creó `tests/conftest.py` con la `fixture` `ventas_validas`.
11. Se utilizó la fixture en una prueba y se registró el marker `unit` en `pyproject.toml`.
12. Se agregó `obtener_ventas_desde_fuente` en `src/ventas.py` para representar una `dependencia externa`.
13. Se utilizaron `Mock`, `return_value` y `assert_called_once_with` para simular la fuente de ventas.
14. Se utilizó `side_effect` para simular un `ConnectionError` y verificar su propagación.

Property-based testing y cobertura:

15. Se creó `tests/test_propiedades.py` y se definió una propiedad sobre la cantidad de ventas y el ingreso total.
16. Se utilizaron estrategias de `Hypothesis` para generar listas, cantidades y precios válidos automáticamente.
17. Se ejecutó la suite completa de pruebas con `pytest`.
18. Se generó un reporte de cobertura en consola con `term-missing`.
19. Se generó un reporte HTML en `htmlcov/`.
20. Se configuró un umbral mínimo de cobertura del 80 %.

Tipado y correcciones de calidad:

21. Se anotaron `Venta`, `Resumen` y `FuenteVentas` mediante `TypedDict` y `Protocol` (contenido del Módulo 5).
22. Se anotaron fixtures y funciones de prueba para satisfacer `mypy` en modo estricto.
23. Se agregaron `pytest` y `hypothesis` como dependencias adicionales del entorno aislado de `mypy`.
24. Se corrigieron los datos de prueba que `mypy` infería como `list[dict[str, object]]`, anotándolos como `list[Venta]`.

Validación y cierre:

25. Se validó el código con las herramientas configuradas anteriormente (Módulos 1 y 5), comprobando `pytest`, `cobertura`, `mypy`, `Ruff`, `Black` y `pre-commit`.
26. `isort` permaneció instalado y configurado, pero no se ejecutó por el bloqueo corporativo documentado.
27. Se creó `.github/workflows/tests.yml` para ejecutar pruebas, `cobertura`, `mypy`, `Ruff` y `Black` en CI.
28. Se actualizó `README.md` con la historia, criterios de aceptación, pruebas, `mocking`, `Hypothesis`, `cobertura` y CI.

29. Se creó el commit final del laboratorio.
30. Se verificó que el repositorio quedara en estado limpio.

RESULTADOS DEL LABORATORIO

Se implementó una historia de ventas mediante TDD, incorporando pruebas unitarias, validaciones, fixtures, parametrización, markers, mocking, property-based testing, cobertura e integración continua.

MÓDULO 02. ACCESO A DATOS Y ORM

CONTENIDO

Modelado de entidades y relaciones con SQLAlchemy Core y ORM. Uso de [sqlite3](#), engines, sesiones, consultas, claves primarias y foráneas, operaciones CRUD y transacciones. Gestión de cambios estructurales mediante Alembic, revisiones y migraciones. Diferencias entre SQLite, PostgreSQL y SQL Server, además de una introducción a bases documentales MongoDB y acceso asíncrono mediante [Motor](#).

LABORATORIO

- Durante la ejecución directa del `src/main.py` no encontraba el paquete `src`, por lo que ejecuté el programa como módulo con `poetry run python -m src.main`.
- Al realizar repetidas pruebas se generaban duplicados, por lo que con cada corrección tuve que eliminar la base de datos `app.bd`, aplicar la migración inicial configurada con alembic y ejecutar de nuevo el programa.
- Inicialmente las pruebas con `mypy` marcaban error por la dependencia alembic, por lo que la agregué como un adicional al hook de mypy dentro del `.pre-commit-config.yaml`.

Preparación del proyecto:

1. Se creó `intermedio/m02-acceso-datos-orm`, se inicializó Poetry, se configuró `package-mode = false`, se preparó `pyproject.toml` y se creó el ambiente virtual.
2. Se seleccionó el intérprete del proyecto, se crearon `src/`, `tests/`, `migrations/`, `versions/`, `README.md` y `.gitignore`, y se instalaron `SQLAlchemy`, `Alembic` y las herramientas de desarrollo (configuración reutilizada de módulos anteriores).

Modelos ORM:

3. Se creó `src/modelos.py` y se definió la clase base declarativa `Base`.
4. Se implementaron los modelos `User`, `Order` y `OrderItem`.
5. Se configuraron claves primarias, campos obligatorios, correo único y fechas de creación.
6. Se definieron las relaciones `User` → `Order` → `OrderItem` con `relationship`, `back_populates` y `cascade`.
7. Se verificó que `Base.metadata` reconociera las tres tablas.

Engine, sesiones y CRUD:

8. Se creó `src/base_datos.py`, se configuró SQLite mediante `engine`, se definió `SessionLocal` y se creó el generador de sesiones.
9. Se creó `src/repositorio.py` con operaciones para crear usuarios, órdenes y elementos, consultar entidades, listar órdenes, actualizar usuarios y eliminar elementos.

10. Se creó **src/main.py**, se integró el flujo de **creación y consulta**, y se ejecutó como módulo mediante `poetry run python -m src.main`.
11. Se corrigieron los imports de `Base`, `SessionLocal` y `engine`, y se resolvió la ejecución desde la raíz del proyecto.
12. Se eliminaron bases locales de prueba cuando fue necesario para evitar conflictos por correos duplicados.

Migraciones con Alembic:

13. Se inicializó Alembic y se generaron **alembic.ini**, **migrations/env.py**, **script.py.mako** y **migrations/versions/**.
14. Se configuró SQLite en **alembic.ini** y se estableció `Base.metadata` como `target_metadata`.
15. Se eliminó `app.db` para generar una migración inicial limpia.
16. Se creó y revisó la migración inicial con las tablas `users`, `orders` y `order_items`, además de sus **claves foráneas**.
17. Se aplicó la **migración** mediante `poetry run alembic upgrade head`.
18. Se verificaron la revisión actual, el historial y la ausencia de cambios pendientes.

Pruebas con SQLite en memoria:

19. Se creó **tests/conftest.py** con una **fixture** que genera un engine SQLite en memoria, crea las tablas y proporciona una sesión aislada.
20. Se creó **tests/test_repositorio.py** con pruebas de creación, consulta, actualización, eliminación y relaciones.
21. Se probó la **restricción** de correo único y el rollback de la sesión después de un `IntegrityError`.
22. Se corrigió la anotación de la **fixture** mediante `Generator[Session, None, None]` y se tiparon las sesiones de las pruebas (tipado estricto del Módulo 5).
23. Se confirmó que la suite pasara con cinco pruebas.

Validación y cierre:

24. Se validó el proyecto con las herramientas configuradas anteriormente (Módulos 1 y 5), comprobando mypy, Ruff, Black y pre-commit.
25. Se añadió Alembic y SQLAlchemy como dependencias adicionales del entorno aislado de mypy.
26. isort permaneció instalado y configurado, pero no se ejecutó por el bloqueo corporativo documentado.

27. Se actualizó **README.md** únicamente con los modelos, relaciones, CRUD, migraciones, transacciones y pruebas de este módulo.
28. Se creó el commit final y se comprobó que el repositorio quedara limpio.

RESULTADOS DEL LABORATORIO

Durante el laboratorio se trabajó con:

- ✓ Usuario inexistente.
- ✓ Orden inexistente.
- ✓ Elemento inexistente.
- ✓ Correo duplicado.
- ✓ Fallos de integridad.
- ✓ Rollback de transacciones.
- ✓ Consultas sin resultados.

Se implementaron:

- ✓ Modelos ORM `User`, `Order` y `OrderItem`.
- ✓ Relaciones uno a muchos.
- ✓ Claves primarias y foráneas.
- ✓ Sesiones SQLAlchemy.
- ✓ Operaciones CRUD.
- ✓ Transacciones y rollback.
- ✓ Migración inicial con Alembic.
- ✓ Pruebas aisladas con SQLite en memoria.
- ✓ Consultas entre entidades relacionadas.
- ✓ Validación de restricciones de base de datos.

Se implementó un sistema de acceso a datos con modelos ORM relacionados, operaciones CRUD, migraciones Alembic, transacciones con rollback y pruebas aisladas sobre SQLite en memoria.

MÓDULO 03. APIS WEB CON FASTAPI**CONTENIDO**

Construcción de APIs web con FastAPI mediante routers, dependencias y esquemas Pydantic. Validación de entradas, respuestas y documentación automática con OpenAPI. Implementación de CRUD, autenticación y autorización mediante JWT, middleware y configuración CORS. Pruebas de endpoints con pytest, aiohttp y bases de datos temporales, incluyendo respuestas HTTP, errores de validación y protección de rutas.

LABORATORIO

- Dado que en recién en el módulo anterior realicé el repositorio remoto para **GitHub**, en esta ocasión **mypy** detectaba módulos duplicados con los archivos **main.py**. Por tanto, se realizó una configuración para **limitar el hook** al módulo correspondiente.
- El endpoint de login no funcionaba con **Swagger OAuth2**, por lo que se reemplazó el modelo JSON por **OAuth2PasswordRequestForm** y se instaló **python-multipart**.
- Se tuvo que fijar **bcrypt==4.0.1** para mantener compatibilidad, pues **Passlib** no podía leer la versión de **bcrypt**.
- Tuve problemas al ejecutar el hook de mypy, pues reportaba funciones de pruebas sin anotaciones. Por ello, se agregaron tipos de parámetros y retornos, incluyendo **TestClient** y **None**.
- Para el entorno aislado de pre-commit se instaló **types-python-jose** y **types-passlib**, porque mypy no encontraba stubs de **JWT** y **Passlib**.

Preparación del proyecto:

1. Se creó *intermedio/m03-fastapi*, se inicializó Poetry, se configuró **package-mode = false**, se prepararon las configuraciones de calidad y se creó el ambiente virtual (configuración reutilizada de módulos anteriores).
2. Se seleccionó el intérprete del proyecto, se crearon *app/*, *app/routers/*, *app/services/*, *tests/*, *README.md* y se utilizó el *.gitignore* raíz del **repositorio consolidado**.
3. Se instalaron **FastAPI**, **Uvicorn**, SQLAlchemy, Alembic, Pydantic, **python-jose**, **Passlib**, **bcrypt**, aiohttp, pytest, pytest-asyncio, pytest-cov, httpx, mypy, Ruff, Black y pre-commit.

Estructura y esquemas:

4. Se crearon **app/main.py**, **dependencies.py**, **models.py**, **schemas.py**, **security.py**, los routers de autenticación y órdenes, y el módulo de servicios.
5. Se definieron los esquemas Pydantic **OrderCreate**, **OrderUpdate**, **OrderOut**, **Token**, **LoginRequest** y **UserOut**.
6. Se configuraron validaciones para productos, cantidades, precios, credenciales y respuestas.

7. Se configuró `from_attributes` para convertir entidades ORM en esquemas de salida.

Aplicación y documentación:

8. Se creó la aplicación `FastAPI` y se registró el endpoint `/health`.
9. Se incorporaron `routers` para autenticación y órdenes.
10. Se verificaron `/health`, `/docs` y `/openapi.json`.
11. Se comprobó la documentación automática mediante `Swagger UI` y `ReDoc`.

CRUD de órdenes:

12. Se implementaron endpoints para crear, listar, consultar, actualizar y eliminar órdenes.
13. Se configuraron códigos de respuesta `201`, `200`, `204`, `404` y `422`.
14. Se probaron órdenes válidas, datos inválidos y órdenes inexistentes.
15. Se asoció cada orden con el usuario autenticado.

Autenticación JWT:

16. Se implementaron funciones para generar, codificar, decodificar y validar `tokens JWT`.
17. Se configuró expiración de tokens y el esquema `Authorization: Bearer`.
18. Se implementó el login mediante `OAuth2PasswordRequestForm`.
19. Se validaron contraseñas mediante `Passlib` y `bcrypt`.
20. Se protegieron los endpoints de órdenes mediante `get_current_user`.
21. Se corrigió la incompatibilidad entre `Passlib` y `bcrypt` fijando `bcrypt==4.0.1`.

Dependencias, base de datos y pruebas:

22. Se configuró `get_db` como dependencia para proporcionar sesiones `SQLAlchemy`.
23. Se resolvió una importación circular causada por la definición de `get_db` después de su uso.
24. Se preparó una base `SQLite temporal` para las pruebas.
25. Se configuró `StaticPool` para compartir la conexión de `SQLite` en memoria.
26. Se sobrescribió `get_db` mediante `app.dependency_overrides`.
27. Se crearon pruebas de login inválido y acceso sin token.
28. Se verificaron respuestas `401`, `404` y `422`.

Calidad y cierre:

29. Se **validó el código** con herramientas configuradas anteriormente (Módulos 1, 2 y 5), comprobando Ruff, Black, mypy y pre-commit.
30. Se agregaron **dependencias adicionales** al hook de **mypy** para analizar FastAPI, SQLAlchemy, Pydantic, JWT y las pruebas.
31. Se corrigió el análisis de mypy para evitar conflictos con módulos **main.py** de otros proyectos del repositorio consolidado.
32. isort permaneció instalado y configurado, pero no se ejecutó por el bloqueo corporativo documentado.
33. Se actualizó **README.md** únicamente con los contenidos de **FastAPI**.
34. Se creó el commit del módulo y se enviaron los cambios al repositorio remoto.

RESULTADOS DEL LABORATORIO

Se implementó una **API FastAPI** que:

- ✓ Expone un endpoint de salud.
- ✓ Valida datos mediante Pydantic.
- ✓ Documenta rutas mediante OpenAPI.
- ✓ Implementa CRUD de órdenes.
- ✓ Gestiona sesiones de base de datos.
- ✓ Protege endpoints mediante JWT.
- ✓ Configura dependencias reutilizables.
- ✓ Utiliza SQLite temporal en pruebas.
- ✓ Comprueba login, validación y autorización mediante pruebas de integración.